

Progetto TPST

Collectiv

Sistema per Collezionisti con
Flutter, PHP e MySQL

Relazione Tecnica di Progetto

Classe 5CII

Girolimetto Riccardo, Muça Alesander, Touel Adem

Anno Scolastico 2025/2026

Indice

1	Introduzione al Progetto	3
1.1	Panoramica	3
1.2	Tecnologie Utilizzate	3
1.2.1	Librerie Flutter	3
1.3	Architettura Generale	4
2	Struttura delle Cartelle	5
2.1	Backend (PHP)	5
2.2	Frontend (Flutter — /collectiv)	5
3	Analisi File per File	6
3.1	Frontend (Flutter)	6
3.1.1	main.dart	6
3.1.2	Models	6
3.1.3	api_service.dart	8
3.1.4	Providers (State Management)	8
3.1.5	app_router.dart	10
3.1.6	Screens	10
3.1.7	Widgets Riutilizzabili	11
3.2	Backend (PHP)	11
3.2.1	.htaccess	11
3.2.2	config.ini	11
3.2.3	index.php — Front Controller	11
3.2.4	config/database.php	12
3.2.5	core/Response.php	12
3.2.6	core/Auth.php	12
3.2.7	api/Auth.php	13
3.2.8	api/Category.php	13
3.2.9	api/Item.php	13
3.2.10	api/Stats.php	14
3.2.11	api/Upload.php	14
3.2.12	api/User.php	15
4	Backend PHP — Dettaglio Architetture	16
5	Documentazione Completa delle API	17
5.1	Autenticazione (/auth)	17
5.2	Utenti (/users)	18
5.3	Categorie (/categories)	18
5.4	Oggetti (/items)	19
5.5	Upload Immagini (/upload)	20
5.6	Statistiche (/stats)	20
6	Database SQL	22

6.1	Tabella <code>users</code>	22
6.2	Tabella <code>categories</code>	22
6.3	Tabella <code>items</code>	23
6.4	Schema Relazionale	23
7	Comunicazione Frontend–Backend	24
7.1	Meccanismo di Scambio Dati	24
7.2	Gestione Autenticazione	24
7.3	Gestione Errori	24
7.4	Esempio Completo: Creazione di una Categoria	24
8	Flusso Completo dell'Applicazione	26
9	Sicurezza e Gestione Errori	27
9.1	Lato Backend (PHP / SQL)	27
9.1.1	Prevenzione SQL Injection	27
9.1.2	Isolamento Multi-Tenant	27
9.1.3	Sicurezza Password	27
9.1.4	Sicurezza Upload	27
9.1.5	Error Handling	27
9.2	Lato Frontend (Flutter)	28
10	Conclusioni	29
10.1	Obiettivo Raggiunto	29
10.2	Criticità Affrontate	29
10.3	Possibili Miglioramenti Futuri	29

Capitolo 1

Introduzione al Progetto

1.1 Panoramica

- **Nome progetto:** Collectiv
- **Obiettivo:** Permettere ai collezionisti di catalogare oggetti di qualsiasi tipo, organizzarli in categorie personali, assegnare voti, aggiungere immagini e consultare classifiche e statistiche.
- **Problema risolto:** Fornisce una piattaforma centralizzata e privata, superando i limiti dei fogli di calcolo o delle app generiche. L'app isola i dati di ciascun utente tramite un'architettura a compartimenti stagni.

1.2 Tecnologie Utilizzate

Livello	Tecnologia
Frontend	Flutter (Dart) — app multiplatforma
Backend	PHP Vanilla — API RESTful senza framework pesanti
Database	MySQL — storage relazionale persistente
Autenticazione	Bearer Token (header <code>Authorization</code>)

Tabella 1.1: Stack tecnologico del progetto.

1.2.1 Librerie Flutter

Pacchetto	Scopo
<code>provider</code>	State Management (<code>ChangeNotifier</code>)
<code>go_router</code>	Navigazione dichiarativa con redirect
<code>http</code>	Richieste HTTP al backend
<code>shared_preferences</code>	Storage locale persistente (token)
<code>google_fonts</code>	Tipografia personalizzata
<code>image_picker</code>	Selezione immagini da galleria/fotocamera

Tabella 1.2: Dipendenze Flutter principali.

1.3 Architettura Generale

Il sistema segue un'architettura **client-server disaccoppiata**. Il frontend comunica col backend scambiando dati in formato JSON su protocollo HTTP. Il backend interroga il database MySQL tramite driver PDO e restituisce il risultato.

Schema architetturale

```
Utente
| interagisce
v
Flutter UI (Schermate)
| chiama metodo su Provider
v
State Management (Provider / ChangeNotifier)
| esegue richiesta
v
HTTP Request (GET / POST / PUT / DELETE) via ApiService
| invia JSON + Bearer Token
v
Backend PHP (index.php --- Front Controller)
| instrada al controller
v
API Controller (es. Item.php)
| query SQL preparata (PDO)
v
Database MySQL (collector_tracker)
| risultato
v
JSON Response
| parsing
v
Flutter Models (fromJson)
| notifyListeners()
v
Aggiornamento reattivo UI Flutter
```

Capitolo 2

Struttura delle Cartelle

2.1 Backend (PHP)

Cartella	Responsabilità
/	Root del progetto; contiene <code>index.php</code> (Front Controller), <code>.htaccess</code> , <code>config.ini</code> e <code>database.sql</code> .
/api	Controller delle risorse REST: <code>Auth.php</code> , <code>User.php</code> , <code>Category.php</code> , <code>Item.php</code> , <code>Stats.php</code> , <code>Upload.php</code> .
/config	<code>database.php</code> — Singleton connessione PDO, legge <code>config.ini</code> .
/core	Helper globali: <code>Auth.php</code> (validazione token), <code>Response.php</code> (formattazione JSON).
/uploads	Creata dinamicamente; storage locale delle immagini caricate.

Tabella 2.1: Struttura cartelle del backend.

2.2 Frontend (Flutter — /collectiv)

Cartella	Responsabilità
lib/	Root sorgente Dart.
lib/models/	Classi dati (DTO): <code>User</code> , <code>Category</code> , <code>Item</code> , <code>FullStats</code> . Responsabili del parsing JSON.
lib/providers/	State management: <code>AuthProvider</code> , <code>CategoryProvider</code> , <code>ItemProvider</code> , <code>StatsProvider</code> .
lib/routing/	Configurazione <code>GoRouter</code> con <code>Auth-Guard</code> .
lib/screens/	Schermate complete: <code>Login</code> , <code>Register</code> , <code>Home</code> , <code>Categories</code> , <code>Items</code> , <code>Stats</code> , <code>Settings</code> .
lib/services/	<code>ApiService</code> — esecuzione fisica delle chiamate HTTP.
lib/widgets/	Widget riutilizzabili: <code>AppLayout</code> , <code>GlassPanel</code> , <code>AnimatedGlassCard</code> , <code>StarRating</code> .

Tabella 2.2: Struttura cartelle del frontend.

Capitolo 3

Analisi File per File

3.1 Frontend (Flutter)

3.1.1 main.dart

Percorso

`collectiv/lib/main.dart`

Scopo

Entry point dell'applicazione. Inizializza i Provider globali e il tema grafico Dark Mode.

Classe

`CollectorTrackerApp (StatefulWidget)`

Variabili principali

`_router` (istanza di `GoRouter`)

Metodo `main()`

- **Input:** nessuno **Output:** void
- Inizializza il binding Flutter (`WidgetsFlutterBinding`).
- Crea e attende `AuthProvider.init()` (carica token da disco).
- Inietta i quattro `ChangeNotifierProvider` nell'albero widget tramite `MultiProvider`.

Metodo `build()`

- **Input:** `BuildContext` **Output:** `Widget`
- Ritorna un `MaterialApp.router` con tema scuro personalizzato (Google Fonts *Plus Jakarta Sans* e *Inter*).

Dipendenze: `provider`, `go_router`, `google_fonts`.

3.1.2 Models

`user.dart`

Classe

`User`

Campi

`id`, `username`, `email`, `apiToken`, `createdAt`

Metodi

`fromJson()` — factory constructor che parse la mappa JSON.
`toJson()` — serializza l'oggetto Dart in mappa.

Utilizzato da

`AuthProvider`

`category.dart`**Classe**

`Category`

Campi

`id`, `name`, `description`, `userId`, `createdAt`

Metodi

`fromJson()`, `toJson()`

Utilizzato da

`CategoryProvider`

`item.dart`**Classe**

`Item`

Campi

`id`, `name`, `description`, `rating`, `acquisitionDate`, `imageUrl`, `userId`, `categoryId`, `createdAt`

Metodi

`fromJson()`, `toJson()`

Utilizzato da

`ItemProvider`

Nota

In `fromJson` l'URL dell'immagine viene arricchito con un timestamp per evitare problemi di cache del browser/device.

`stats.dart`**Classi presenti**

`FullStats`, `StatsOverview`, `CategoryStat`, `RatingBucket`, `TimelineEntry`, `StatsRecords`, `RecordItem`, `BestCategory`

Scopo

Modellare l'intera risposta JSON complessa dell'endpoint `/stats`, comprensiva di overview, distribuzione voti, timeline acquisizioni e record personali.

Utilizzato da

`StatsProvider`

3.1.3 `api_service.dart`

Percorso

`collectiv/lib/services/api_service.dart`

Scopo

Gestione centralizzata di **tutte** le chiamate HTTP verso il backend.

Classi

`ApiService`, `ApiException`

Variabile

`baseUrl = http://localhost/web`

Metodo `_getHeaders()` Legge il token da `SharedPreferences`; se presente, aggiunge l'header `Authorization: Bearer <token>`.

Metodi `get()`, `post()`, `put()`, `delete()`

- **Input:** `endpoint` (String), `body` (Map, solo per POST/PUT)
- **Output:** `Future<dynamic>`
- Effettuano la richiesta di rete con il pacchetto `http` e invocano `_handleResponse()`.

Metodo `uploadMultipart()` Invia file in formato `multipart/form-data` per l'upload delle immagini.

Metodo `_handleResponse()` Se `statusCode` è 200–299, decodifica il JSON; altrimenti lancia `ApiException`.

Dipendenze: `http`, `shared_preferences`, `dart:convert`.

3.1.4 Providers (State Management)

`auth_provider.dart`

Classe

`AuthProvider` (extends `ChangeNotifier`)

Stato

`_currentUser`, `_isAuthenticated`, `_isLoading`

- **`init()`:** Cerca il token su disco; se presente, chiama `fetchMe()`.
- **`login(email, password)`:** POST a `/auth/login`, salva token, chiama `fetchMe()`, setta `_isAuthenticated = true`.
- **`register(username, email, password)`:** POST a `/auth/register`.
- **`updateProfile(data)`:** PUT a `/users/{id}`.
- **`deleteAccount()`:** DELETE a `/users/{id}`, poi `logout()`.
- **`logout()`:** Rimuove token, resetta stato, `notifyListeners()`.

Utilizzato da: LoginScreen, RegisterScreen, SettingsScreen, AppRouter.

`category_provider.dart`

Classe

CategoryProvider (extends ChangeNotifier)

Stato

`_categories` (List), `_isLoading`

- `fetchCategories()`: GET /categories; popola la lista locale.
- `createCategory(name, description)`: POST /categories; poi refresh.
- `updateCategory(id, name, desc)`: PUT /categories/{id}.
- `deleteCategory(id)`: DELETE /categories/{id}.

`item_provider.dart`

Classe

ItemProvider (extends ChangeNotifier)

Stato

`_allItems`, `_currentPage`, `_hasMore`, `_searchQuery`, `_categoryId`

- `fetchItems(refresh, search)`: GET /items?page=X&limit=20&search=.... Gestisce la paginazione accumulando i risultati. Aggiorna `_hasMore` dal campo `meta.total_pages`.
- `fetchSingleItem(id)`: GET /items/{id} per i dettagli completi.
- `createItem(data)`: POST /items.
- `updateItem(id, data)`: PUT /items/{id}.
- `deleteItem(id)`: DELETE /items/{id}.
- `uploadImage(id, file)`: Legge i byte dell'immagine e chiama `uploadMultipart()`.
- `setCategoryFilter(id)`: Filtra la lista in-memory per categoria.

`stats_provider.dart`

Classe

StatsProvider (extends ChangeNotifier)

Stato

`_fullStats` (FullStats)

- `fetchGlobalStats()`: GET /stats; parsata in `FullStats.fromJson()`.
- `fetchCategoryStats(categoryId)`: GET /stats/{category_id}.

3.1.5 app_router.dart

Percorso

`collectiv/lib/routing/app_router.dart`

Scopo

Definire la navigazione con `GoRouter`, incluso l'Auth-Guard.

Funzione `createRouter(AuthProvider)`

- Parametro `redirect`: se `isAuthenticated == false` e la rotta non è `/login` né `/register`, forza il redirect a `/login`.
- Usa una `ShellRoute` con `AppLayout` come wrapper grafico (drawer, menu laterale) attorno a tutte le pagine autenticate.
- Rotte definite: `/` (Home), `/categories`, `/items`, `/stats`, `/settings`.

3.1.6 Screens

Le schermate sono implementate come `StatefulWidget` e utilizzano i `Provider` tramite `context.read<T>()` e `context.watch<T>()` per reagire ai cambiamenti di stato.

Screen	Descrizione
<code>login_screen.dart</code>	Form di login; invoca <code>AuthProvider.login()</code> .
<code>register_screen.dart</code>	Form di registrazione; invoca <code>AuthProvider.register()</code> .
<code>home_screen.dart</code>	Dashboard principale con statistiche rapide.
<code>categories_screen.dart</code>	CRUD categorie con dialog di creazione/modifica.
<code>items_screen.dart</code>	Lista paginata, ricerca, creazione/modifica/eliminazione oggetti, upload immagini.
<code>stats_screen.dart</code>	Visualizzazione statistiche avanzate (grafici, record, distribuzione voti).
<code>settings_screen.dart</code>	Modifica profilo, cambio password, eliminazione account, logout.

Tabella 3.1: Elenco delle schermate Flutter.

Flusso tipico (Login)

1. L'utente compila email e password nei `TextField`.
2. Preme il pulsante "Accedi".
3. Flutter chiama `await AuthProvider.login(email, password)`.
4. Se successo: `GoRouter` rileva `isAuthenticated = true` e reindirige a `/`.

3.1.7 Widgets Riutilizzabili

Widget	Descrizione
<code>app_layout.dart</code>	Layout principale con drawer/navigation bar. Avvolge tutte le schermate autenticate.
<code>glass_panel.dart</code>	Pannello con effetto glassmorphism.
<code>animated_glass_card.dart</code>	Card animata con effetti di hover e transizione.
<code>star_rating.dart</code>	Widget per selezionare e visualizzare il rating a stelle.

Tabella 3.2: Widget personalizzati.

3.2 Backend (PHP)

3.2.1 .htaccess

Scopo

Riscrittura URL Apache: tutte le richieste che non corrispondono a file o directory fisiche vengono instradate a `index.php?url=...`

```

1 RewriteEngine On
2 RewriteCond %{REQUEST_FILENAME} !-f
3 RewriteCond %{REQUEST_FILENAME} !-d
4 RewriteRule ^(.*)$ index.php?url=$1 [QSA,L]
```

Listing 3.1: Contenuto del file .htaccess

3.2.2 config.ini

Scopo

Parametri di configurazione del database isolati dal codice sorgente (non versionato su Git per sicurezza).

Contenuto

Sezione [database] con host, dbname, user, password.

3.2.3 index.php — Front Controller

Scopo

Unico punto d'ingresso di tutte le richieste HTTP. Analizza l'URL e instrada al controller appropriato.

Variabili

`$url`, `$resource`, `$method`, `$pdo`

Funzionamento

1. Carica `config/database.php`, `core/Response.php`, `core/Auth.php`.
2. Inizializza la connessione PDO (`getDbConnection()`) in un blocco `try/catch`.

3. Estrae `$resource` dalla URL (primo segmento: `auth`, `items`, ecc.).
4. Uno `switch($resource)` include il controller PHP corrispondente dalla cartella `/api/` e invoca la funzione `handle*()` passando `$pdo`, `$method` e l'eventuale parametro ID.

3.2.4 `config/database.php`

Scopo

Connessione al database MySQL tramite PDO con pattern Singleton.

Funzione `getDbConnection()`

- Legge il file `config.ini` con `parse_ini_file()`.
- Crea una connessione PDO con charset `utf8mb4`.
- Configura `ERRMODE_EXCEPTION`, `FETCH_ASSOC` e `EMULATE_PREPARES = false` per massima sicurezza.
- Pattern **Singleton**: la variabile statica `$pdo` garantisce una sola connessione per richiesta.

3.2.5 `core/Response.php`

Scopo

Standardizzare tutte le risposte JSON dell'API.

- `sendResponse($statusCode, $data)`: Imposta header `Content-Type: application/json`, stampa `json_encode()` e termina lo script con `exit`.
- `sendError($statusCode, $message)`: Genera un formato di errore standardizzato ispirato a RFC 7807: `{"error": {"code": 400, "message": "..."} }`.
- `getJsonInput()`: Legge il body della richiesta da `php://input` e lo decodifica in array associativo.

3.2.6 `core/Auth.php`

Scopo

Validazione del Bearer Token per gli endpoint protetti.

Funzione `requireUserAuth($pdo)`

1. Legge gli header HTTP con `getallheaders()`.
2. Cerca l'header `Authorization` (case-insensitive).
3. Estrae il token con regex: `/Bearer\s(\S+)/`.
4. Esegue `SELECT id, username, email FROM users WHERE api_token = :token`.
5. Se il token non corrisponde a nessun utente: `sendError(401)`.
6. Se valido: ritorna l'array associativo con i dati dell'utente.

3.2.7 api/Auth.php

Scopo

Gestione Registrazione e Login.

Funzione register(\$pdo)

1. Legge JSON body: `username`, `email`, `password`.
2. Hash della password con `password_hash()` e algoritmo BCRYPT.
3. INSERT in tabella `users`; gestione errore 1062 (duplicato) con codice HTTP 409.

Funzione login(\$pdo)

1. Legge `email` e `password` dal JSON body.
2. SELECT dell'utente per email.
3. Verifica password con `password_verify()`.
4. Genera token sicuro: `bin2hex(random_bytes(32))`.
5. UPDATE del campo `api_token` sulla riga utente.
6. Ritorna JSON con `token` e dati profilo.

3.2.8 api/Category.php

Scopo

CRUD completo delle categorie personali.

Funzione

`handleCategories($pdo, $method, $id)`

- **GET**: Se `$id` è presente, ritorna la singola categoria; altrimenti ritorna tutte le categorie dell'utente (`WHERE user_id = :uid`).
- **POST**: Inserisce una nuova categoria con validazione nome obbligatorio e gestione duplicato (`UNIQUE constraint`).
- **PUT/PATCH**: Aggiornamento dinamico (solo campi forniti) con whitelist [`'name'`, `'description'`].
- **DELETE**: Cancella la categoria; gli item associati vengono eliminati in cascata (`ON DELETE CASCADE`).

3.2.9 api/Item.php

Scopo

CRUD completo degli oggetti della collezione.

Funzione

`handleItems($pdo, $method, $id)`

- **GET (lista):** Implementa **paginazione** con LIMIT e OFFSET dinamici e **ricerca** con LIKE. Una seconda query COUNT(*) genera i metadata (total_items, total_pages, current_page).
- **GET (singolo):** SELECT * con clausola di sicurezza AND user_id = :uid.
- **POST:** Inserisce l'oggetto verificando prima che la category_id appartenga all'utente loggato. Il campo user_id è **forzato** dal token, mai dal body.
- **PUT/PATCH:** Aggiornamento dinamico con whitelist.
- **DELETE:** Rimozione con doppia clausola WHERE id = :id AND user_id = :uid.

3.2.10 api/Stats.php

Scopo

Generazione di statistiche avanzate aggregate.

Funzione

handleStats(\$pdo, \$method, \$categoryId)

L'endpoint accetta un filtro opzionale \$categoryId dalla rotta /stats/{id}. Le sezioni della risposta JSON sono:

1. **Overview:** COUNT(*), AVG(rating), STDDEV(rating), conteggio item con rating 10, item con immagine, item degli ultimi 30 giorni. Formula personalizzata “*Collector Score*”.
2. **By Category:** LEFT JOIN tra categories e items con GROUP BY per distribuzione.
3. **Rating Distribution:** GROUP BY rating per la curva dei voti.
4. **Acquisition Timeline:** GROUP BY YEAR(acquisition_date) per la cronologia annuale.
5. **Records:** Item con voto più alto, acquisizione più vecchia, più recente, e categoria migliore.

3.2.11 api/Upload.php

Scopo

Caricamento e cancellazione immagini associate agli item.

Funzione

handleUpload(\$pdo, \$method, \$itemId)

POST (Upload)

1. Verifica che l'item esista e appartenga all'utente loggato.
2. Controlla il MIME type con finfo_file() (solo JPEG, PNG, WEBP).
3. Limite dimensione: 5 MB.
4. Genera un nome file univoco con uniqid() per evitare collisioni.
5. Salva il file con move_uploaded_file() nella cartella /uploads/items/.

6. Se esisteva un'immagine precedente, la elimina dal disco.
7. Aggiorna il campo `image_url` dell'item nel database.

DELETE Elimina il file fisico dal disco e imposta `image_url = NULL` nel database.

3.2.12 `api/User.php`

Scopo

Gestione profilo utenti.

Funzione

`handleUser($pdo, $method, $id)`

- **GET**: Lista di tutti gli utenti (dati parziali) o dettaglio di uno specifico.
- **PUT**: Aggiornamento **solo del proprio profilo** (username, email, password). Controllo `(int)$id !== (int)$userId` per impedire modifiche ad altri account.
- **DELETE**: Eliminazione definitiva **solo del proprio account**. Le FK in cascata eliminano tutti i dati associati.

Capitolo 4

Backend PHP — Dettaglio Architet- turale

Il backend segue il pattern **Front Controller**: tutte le richieste HTTP arrivano a `index.php`, che le smista ai controller nella cartella `/api/`. Non vengono utilizzati framework esterni: l'architettura è in **PHP Vanilla puro** per massime prestazioni e trasparenza didattica.

Il backend è **Stateless**: non utilizza le sessioni PHP tradizionali (`$_SESSION`), ma riconosce gli utenti esclusivamente tramite il Bearer Token inviato nell'header `Authorization`.

Flusso di una richiesta generica

1. La richiesta HTTP arriva al web server Apache.
2. `.htaccess` riscrive l'URL verso `index.php?url=...`
3. `index.php` estrae la risorsa (`$resource`) e il metodo HTTP.
4. Lo `switch` carica il controller appropriato e invoca `handle*()`.
5. Il controller chiama `requireUserAuth()` per validare il token.
6. Viene eseguita la logica di business con query PDO preparate.
7. La risposta JSON viene inviata tramite `sendResponse()`.

Capitolo 5

Documentazione Completa delle API

Tutti gli endpoint, salvo `/auth/login` e `/auth/register`, richiedono l'header:

`Authorization: Bearer <token>`

5.1 Autenticazione (`/auth`)

POST `/auth/register`

Scopo: Registra un nuovo utente.

Body JSON: `{"username": "...", "email": "...", "password": "..."}"`

Response (201): `{"message": "Registrazione completata.", "id": 2}`

Errori: 400 (campi mancanti), 409 (username/email già in uso).

File PHP: `api/Auth.php` **Tabella:** `users` (INSERT)

Frontend: `AuthProvider.register()`

POST `/auth/login`

Scopo: Autentica l'utente e restituisce il Bearer Token.

Body JSON: `{"email": "...", "password": "..."}"`

Response (200):

```
1 {
2   "message": "Login completato con successo.",
3   "token": "a1b2c3d4e5f6...",
4   "user": {"id": 1, "username": "admin", "email": "
           admin@mail.com"}
5 }
```

Errori: 400 (campi mancanti), 401 (credenziali errate).

File PHP: `api/Auth.php` **Tabella:** `users` (SELECT, UPDATE)

Frontend: `AuthProvider.login()`

GET `/auth/me`

Scopo: Ritorna i dati dell'utente proprietario del token corrente.

Response (200): `{"user": {"id": 1, "username": "...", "email": "..."}}`

Errori: 401.

Frontend: `AuthProvider.fetchMe()`

5.2 Utenti (/users)

GET /users

Scopo: Lista parziale di tutti gli utenti registrati.

Response (200): [{"id": 1, "username": "admin"}, ...]

GET /users/{id}

Scopo: Dati completi di un utente specifico.

Response (200): {"id": 1, "username": "...", "email": "...", "created_at": "..."}
Errori: 404.

PUT /users/{id}

Scopo: Aggiorna il proprio profilo (username, email, password).

Vincolo: L'ID nella rotta deve corrispondere all'utente loggato.

Errori: 403 (modifica profilo altrui), 409 (username/email duplicato).

DELETE /users/{id}

Scopo: Elimina definitivamente il proprio account e tutti i dati associati.

Vincolo: Solo il proprio account.

5.3 Categorie (/categories)

GET /categories

Scopo: Lista delle categorie dell'utente loggato.

Response (200): [{"id": 1, "name": "Videogames"}, ...]

GET /categories/{id}

Scopo: Dettaglio completo di una categoria.

Errori: 404 (non trovata o non autorizzata).

POST /categories

Scopo: Crea una nuova categoria personale.

Body JSON: {"name": "RetroGaming", "description": "Giochi PS1"}

Response (201): {"message": "...", "id": 5}

Errori: 400 (nome mancante), 409 (duplicato).

PUT /categories/{id}

Scopo: Modifica nome e/o descrizione.

Errori: 404, 409.

DELETE /categories/{id}

Scopo: Elimina la categoria e **tutti** gli item associati (CASCADE).

Errori: 404.

5.4 Oggetti (/items)

GET /items

Scopo: Elenco paginato e ricercabile degli oggetti dell'utente.

Query params: ?page=1&limit=20&search=valore

Response (200):

```
1 {
2   "data": [{"id": 1, "name": "...", "rating": 8, ...}],
3   "meta": {
4     "current_page": 1,
5     "per_page": 20,
6     "total_items": 85,
7     "total_pages": 5
8   }
9 }
```

Frontend: ItemProvider.fetchItems()

GET /items/{id}

Scopo: Dettaglio completo di un singolo oggetto (inclusi description, acquisition_date).

Frontend: ItemProvider.fetchSingleItem()

POST /items

Scopo: Crea un nuovo oggetto nella collezione personale.

Body JSON: {"name": "...", "category_id": 1, "rating": 10, "description": "..."}
Validazione: Verifica che category_id appartenga all'utente.

PUT /items/{id}

Scopo: Aggiornamento campi dell'oggetto.

Campi aggiornabili: name, description, rating, acquisition_date, category_id.

DELETE /items/{id}**Scopo:** Rimuove l'oggetto dalla collezione.**Errori:** 404 (non trovato o non autorizzato).

5.5 Upload Immagini (/upload)

POST /upload/{item_id}**Scopo:** Carica un'immagine e la associa all'oggetto.**Content-Type:** multipart/form-data**Campo form:** image (file)**Formati accettati:** JPEG, PNG, WEBP **Dimensione max:** 5 MB**Response (201):** {"message": "...", "image_url": "/uploads/items/..."}**Frontend:** ItemProvider.uploadImage()**DELETE /upload/{item_id}****Scopo:** Elimina l'immagine dal server e rimuove il riferimento dal database.

5.6 Statistiche (/stats)

GET /stats**Scopo:** Statistiche globali dell'intera collezione dell'utente.**Response (200):**

```

1  {
2    "overview": {
3      "collector_score": 1400,
4      "total_items": 85,
5      "average_rating": 7.3,
6      "grading_consistency": 1.8,
7      "perfect_items": 12,
8      "items_with_images": 45,
9      "items_last_30_days": 5,
10     "total_categories": 8
11   },
12   "by_category": [{"id":1, "category":"Videogames", "count":30, ...}],
13   "rating_distribution": [{"rating":10, "count":12}, ...],
14   "acquisition_timeline": [{"year":2020, "count":15}, ...],
15   "records": {
16     "topRated": {"id":42, "name":"...", "rating":10},
17     "oldest_acquisition": {...},
18     "newest_acquisition": {...},
19     "best_category": {"name":"...", "avg_rating":9.2}

```

```
20   }  
21 }
```

Frontend: `StatsProvider.fetchGlobalStats()`

GET `/stats/{category_id}`

Scopo: Statistiche filtrate per una singola categoria.

Differenze: I campi `by_category` e `best_category` non sono presenti.

Frontend: `StatsProvider.fetchCategoryStats()`

Flusso completo di chiamata API (esempio: paginazione oggetti)

1. Flutter (`ItemsScreen`) rileva lo scroll a fine lista.
2. `ItemProvider` costruisce la query `?page=2&limit=20`.
3. `ApiService` esegue HTTP GET con header Bearer Token.
4. PHP (`index.php`) instrada a `Item.php`.
5. `requireUserAuth()` valida il token e identifica l'utente.
6. Query SQL: `SELECT ... LIMIT 20 OFFSET 20` sulla tabella `items`.
7. PDO restituisce l'array → PHP codifica con `json_encode()`.
8. JSON inviato come response HTTP.
9. `ApiService` decodifica il JSON.
10. `ItemProvider` istanzia gli oggetti Dart con `Item.fromJson()`.
11. `notifyListeners()` → la `ListView` Flutter si aggiorna.

Capitolo 6

Database SQL

Il database si chiama `collector_tracker` e utilizza il charset `utf8mb4` per il supporto completo a tutti i caratteri Unicode. L'architettura prevede **3 tabelle relazionali**, progettate per garantire l'isolamento Multi-Tenant dei dati.

6.1 Tabella `users`

Campo	Tipo	Descrizione
<code>id</code>	INT (PK, AI)	Identificativo univoco auto-incrementante.
<code>username</code>	VARCHAR(50) UNIQUE	Nome utente visibile.
<code>email</code>	VARCHAR(100) UNIQUE	Email per l'accesso.
<code>password_hash</code>	VARCHAR(255)	Hash bcrypt della password.
<code>api_token</code>	VARCHAR(128) UNIQUE	Bearer Token generato al login.
<code>created_at</code>	TIMESTAMP	Data di registrazione.

Tabella 6.1: Struttura tabella `users`.

6.2 Tabella `categories`

Campo	Tipo	Descrizione
<code>id</code>	INT (PK, AI)	Identificativo univoco.
<code>name</code>	VARCHAR(100)	Nome della categoria.
<code>description</code>	TEXT	Descrizione opzionale.
<code>user_id</code>	INT (FK)	Riferimento a <code>users.id</code> .
<code>created_at</code>	TIMESTAMP	Data di creazione.

Tabella 6.2: Struttura tabella `categories`.

Vincoli: FOREIGN KEY (`user_id`) REFERENCES `users(id)` ON DELETE CASCADE.

Constraint UNIQUE: (`name`, `user_id`) — impedisce categorie duplicate per lo stesso utente.

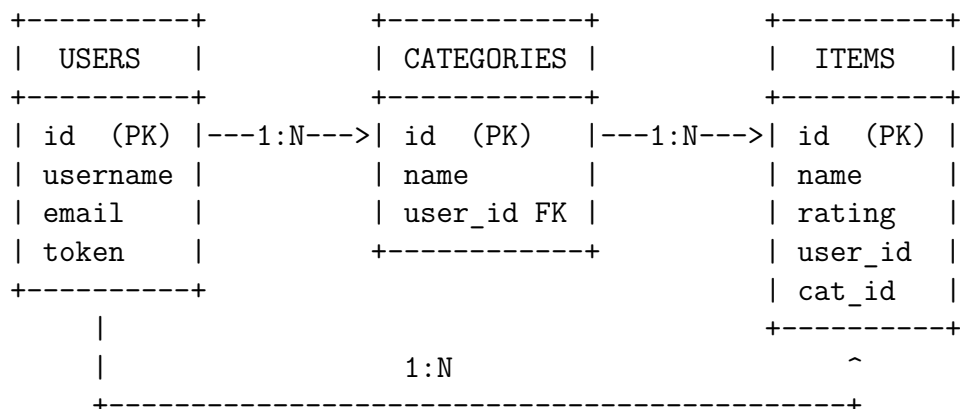
6.3 Tabella items

Campo	Tipo	Descrizione
id	INT (PK, AI)	Identificativo univoco.
name	VARCHAR(150)	Nome dell'oggetto.
description	TEXT	Descrizione dettagliata.
rating	INT	Voto da 1 a 10 (CHECK constraint).
acquisition_date	DATE	Data di acquisizione.
image_url	VARCHAR(255)	Percorso relativo dell'immagine caricata.
user_id	INT (FK)	Riferimento a users.id.
category_id	INT (FK)	Riferimento a categories.id.
created_at	TIMESTAMP	Data di inserimento.

Tabella 6.3: Struttura tabella items.

Vincoli FK: Doppia chiave esterna con ON DELETE CASCADE sia verso users sia verso categories.

6.4 Schema Relazionale



Regola ON DELETE CASCADE

Se un utente viene eliminato, **tutte** le sue categorie e i suoi item vengono rimossi automaticamente dal database grazie alle chiavi esterne in cascata. Lo stesso vale per l'eliminazione di una categoria: tutti gli item ad essa associati vengono distrutti.

Capitolo 7

Comunicazione Frontend–Backend

Flutter agisce come **Client HTTP** e PHP come **Server API**. La comunicazione è interamente **asincrona** (`async/await`) per evitare il blocco della UI.

7.1 Meccanismo di Scambio Dati

1. **Codifica (Dart → PHP):** La Map Dart viene serializzata in stringa JSON tramite `jsonEncode()` e inviata come body della richiesta HTTP.
2. **Ricezione (PHP):** PHP legge il body grezzo da `php://input` con `file_get_contents()` e lo decodifica con `json_decode()` in un array associativo.
3. **Risposta (PHP → Dart):** PHP stampa la risposta con `json_encode()` e header `Content-Type: application/json`.
4. **Decodifica (Dart):** `ApiService` decodifica il JSON con `jsonDecode()` e lo passa ai modelli tramite factory constructor `fromJson()`.

7.2 Gestione Autenticazione

Al login, il token viene salvato in `SharedPreferences` (storage persistente del dispositivo). Ad ogni richiesta successiva, `ApiService._getHeaders()` lo recupera e inietta l'header:

```
Authorization: Bearer <TOKEN>
```

7.3 Gestione Errori

Se il server risponde con `statusCode >= 300`, `ApiService` lancia una `ApiException`. I Provider la intercettano nei blocchi `try/catch` e l'UI mostra una `SnackBar` con il messaggio di errore.

7.4 Esempio Completo: Creazione di una Categoria

1. L'utente clicca "Salva" nel form categoria in Flutter.
2. Flutter esegue `CategoryProvider.createCategory('Funko', 'Pupazzi')`.
3. Il Provider invoca `ApiService.post('/categories', {'name': 'Funko', ...})`.
4. `ApiService` recupera il token, formatta il body in JSON, invia HTTP POST.
5. PHP (`index.php`) instrada a `Category.php`.

6. `requireUserAuth()` valida il token e identifica `user_id = 1`.
7. PHP esegue: `INSERT INTO categories (name, user_id) VALUES ('Funko', 1)`.
8. PDO completa la query. PHP invia `{"message": "Categoria creata"}` con codice 201.
9. Flutter riceve la risposta (`await`).
10. Il Provider ricarica l'elenco locale (`fetchCategories()`).
11. L'UI si aggiorna reattivamente grazie a `notifyListeners()`.

Capitolo 8

Flusso Completo dell'Applicazione

1. **Apertura App:** `main.dart` avvia `CollectorTrackerApp`. Viene invocato `AuthProvider.init()`, che cerca un token salvato in `SharedPreferences`.
2. **Auth-Guard:** Se il token manca o è invalido, `GoRouter` reindirige forzatamente su `/login`. L'utente vede la schermata di Login.
3. **Login:** L'utente inserisce le credenziali. Richiesta POST a `/auth/login`. Il token viene salvato. `GoRouter` rileva `isAuthenticated = true` e reindirige sulla Home.
4. **Dashboard / AppLayout:** L'interfaccia principale mostra un menu laterale (drawer) con le sezioni: Home, Categorie, Items, Statistiche, Impostazioni.
5. **Visualizzazione Oggetti:** L'utente apre "Items". Flutter chiama `GET /items?page=1`. Mostra una lista/griglia.
6. **Paginazione (Lazy Loading):** Lo scroll a fine lista lancia automaticamente la richiesta per `page=2`, `page=3`, ecc.
7. **Creazione Oggetto:** L'utente preme "+", compila il form (nome, categoria, voto) e salva. POST a `/items`. L'elenco si aggiorna.
8. **Upload Immagine:** Dall'oggetto creato, l'utente seleziona un'immagine dalla galleria. Invio multipart a `POST /upload/{id}`.
9. **Statistiche:** GET su `/stats` genera i dati aggregati. L'UI mostra Collector Score, distribuzione voti, timeline e record.
10. **Logout:** L'utente preme "Esci". Il token viene rimosso da `SharedPreferences`. `GoRouter` reindirige su `/login`.

Capitolo 9

Sicurezza e Gestione Errori

9.1 Lato Backend (PHP / SQL)

9.1.1 Prevenzione SQL Injection

Tutte le query utilizzano **PDO Prepared Statements**. L'input utente non viene mai concatenato direttamente nelle query SQL:

```
1 $stmt = $pdo->prepare(  
2     "SELECT * FROM items WHERE id = :id AND user_id = :uid"  
3 );  
4 $stmt->execute(['id' => $id, 'uid' => $userId]);
```

Listing 9.1: Esempio di Prepared Statement

I segnaposto (:id, :uid) vengono trattati come dati puri, mai come codice SQL eseguibile.

9.1.2 Isolamento Multi-Tenant

Ogni query di tipo UPDATE, DELETE e SELECT porta sempre la clausola **AND user_id = :uid**. L'ID utente viene ricavato **esclusivamente** dalla validazione server-side del token, mai dal body della richiesta.

9.1.3 Sicurezza Password

Le password sono salvate con `password_hash()` usando l'algoritmo **BCRYPT** con sale automatico. La verifica avviene con `password_verify()`. Le password in chiaro non esistono mai nel database.

9.1.4 Sicurezza Upload

- Validazione MIME type reale con `finfo_file()` (non basata sull'estensione).
- Whitelist formati: solo JPEG, PNG, WEBP.
- Limite dimensione: 5 MB.
- Nome file generato casualmente con `uniqid()` per prevenire sovrascritture e path traversal.

9.1.5 Error Handling

Blocchi `try/catch` su ogni endpoint mascherano gli errori interni (codice 500 generico al posto di stack trace PHP).

9.2 Lato Frontend (Flutter)

- **Validazione form:** I `TextFormField` hanno validatori che inibiscono il submit in caso di dati incompleti.
- **Gestione HTTP:** Risposte con status code ≥ 300 vengono convertite in `ApiException` e catturate dai Provider per mostrare `SnackBar` informative.
- **Token persistente:** Il token è salvato in `SharedPreferences`, una soluzione sicura per la persistenza locale su dispositivi mobili.

Capitolo 10

Conclusioni

10.1 Obiettivo Raggiunto

Il progetto Collectiv implementa con successo un ecosistema multiplatforma completo, dimostrando padronanza delle architetture client-server REST, dello state management reattivo in Flutter (Provider Pattern) e della progettazione di API RESTful sicure in PHP.

10.2 Criticità Affrontate

- Progettazione di un sistema a compartimenti stagni per l'isolamento completo dei dati utente.
- Implementazione di una paginazione efficiente con metadata, accoppiata alla UI con lazy loading.
- Gestione sicura dell'upload di file con validazione MIME e naming univoco.
- Reattività dell'interfaccia Flutter grazie al pattern Provider/ChangeNotifier.

10.3 Possibili Miglioramenti Futuri

1. **Cache offline:** Implementazione di SQLite lato Flutter (`sqflite`) per permettere la consultazione dei dati anche senza connessione.
2. **Object Storage cloud:** Migrazione dello storage immagini verso servizi come AWS S3 per scalabilità.
3. **Autenticazione avanzata:** Adozione di JWT (JSON Web Tokens) e flussi OAuth 2.0 per supportare “Login con Google” o “Login con Facebook”.
4. **Testing automatizzato:** Aggiunta di unit test per i Provider Flutter e test di integrazione per gli endpoint PHP.
5. **Caching server-side:** Implementazione di Redis per ridurre il carico sul database nelle query statistiche complesse.